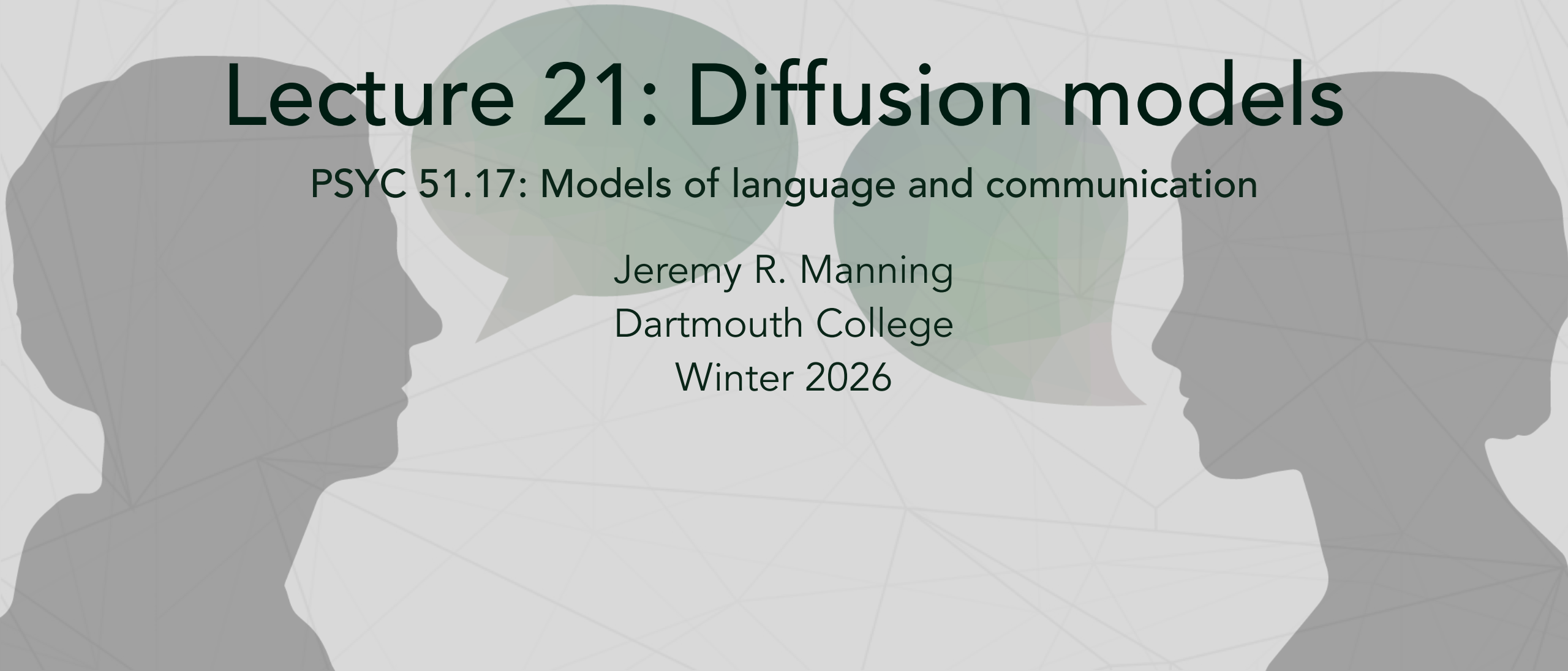


Lecture 21: Diffusion models

PSYC 51.17: Models of language and communication

Jeremy R. Manning
Dartmouth College
Winter 2026



Learning objectives

By the end of this lecture, you will be able to...

1. Explain the **forward diffusion process** and how it progressively destroys information
2. Derive why the **reverse process** requires a learned neural network
3. Describe the **U-Net architecture** for noise prediction with timestep conditioning
4. Understand the **simplified DDPM training objective** and why it works
5. Compare **DDPM** and **DDIM** sampling strategies and their tradeoffs

Announcements

Assignment 4 due today!

Assignment 4 (Customer Service Chatbot) is due **today at 11:59 PM EST.**

Final project and optional assignment

- **Final Project** has been released! Due **March 9, 11:59 PM EST**
- **Assignment 5** (Build GPT) is now available as **optional extra credit**

Why diffusion models?

A different approach to generation

Autoregressive models like transformers generate one token at a time, left to right. Diffusion models take a fundamentally different approach: they generate **everything at once** through iterative refinement.

The core idea

What if instead of building an output one piece at a time, we started with **pure noise** and gradually refined it into something meaningful? This is the intuition behind diffusion models — and it has produced some of the most stunning generative AI results to date (DALL-E, Stable Diffusion, Sora).

The thermodynamic inspiration

Sohl-Dickstein et al. (2015)

Diffusion models are inspired by **non-equilibrium thermodynamics**. Imagine dropping a drop of ink into water:

- **Forward process:** The ink disperses until the water is uniformly colored (order → disorder)
- **Reverse process:** If we could reverse time, the uniform color would reconcentrate into the original drop

The key insight from Sohl-Dickstein et al. (2015): if the forward process is simple enough (small Gaussian noise at each step), then the reverse process is **also approximately Gaussian** — and we can learn it with a neural network.

The forward diffusion process

Progressively adding noise

Given a clean data sample $\mathbf{x}_0$, the forward process adds a small amount of Gaussian noise at each timestep $t = 1, 2, \dots, T$:

$$q(\mathbf{x}_t | \mathbf{x}_{t-1}) = \mathcal{N}(\mathbf{x}_t; \sqrt{1 - \beta_t} \mathbf{x}_{t-1}, \beta_t \mathbf{I})$$

where β_t is a small positive constant (the **noise schedule**) that controls how much noise is added at each step.

What this means

At each step, we slightly shrink the signal ($\sqrt{1 - \beta_t}$) and add some noise (β_t). After enough steps ($T \approx 1000$), the original data is completely destroyed — $\mathbf{x}_T$ is indistinguishable from pure Gaussian noise.

The forward process

Noise schedules

How fast should we add noise?

The noise schedule $\{\beta_1, \beta_2, \dots, \beta_T\}$ controls the rate of information destruction. We track the **cumulative signal remaining** as $\bar{a}_t = \prod_{s=1}^t (1 - \beta_s)$:

- **Linear schedule** (Ho et al., 2020): β_t increases linearly from $\beta_1 = 10^{-4}$ to $\beta_T = 0.02$
- **Cosine schedule** (Nichol & Dhariwal, 2021): Designed so that $\bar{a}_t$ follows a cosine curve, preserving more signal at early timesteps

Why the schedule matters

A linear schedule destroys too much information too quickly in the early steps. The cosine schedule preserves fine details longer, leading to better sample quality — especially for high-resolution images.

Noise schedules

The closed-form shortcut

Jumping directly to any timestep

We don't need to apply noise one step at a time. Define $a_t = 1 - \beta_t$ and $\bar{a}_t = \prod_{s=1}^t a_s$. Then we can jump directly from $\mathbf{x}_0$ to any $\mathbf{x}_t$:

$$q(\mathbf{x}_t | \mathbf{x}_0) = \mathcal{N}(\mathbf{x}_t; \sqrt{\bar{a}_t} \mathbf{x}_0, (1 - \bar{a}_t) \mathbf{I})$$

Reparameterization trick

In practice, we sample $\mathbf{x}_t$ using:

$$\mathbf{x}_t = \sqrt{\bar{a}_t} \mathbf{x}_0 + \sqrt{1 - \bar{a}_t} \boldsymbol{\epsilon}, \quad \boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$$

This is essential for training — we can generate any noisy version of $\mathbf{x}_0$ in a single step, without iterating through all t steps.

The reverse process

Learning to denoise

The reverse process starts from noise $\mathbf{x}_T \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ and gradually removes noise to recover $\mathbf{x}_0$:

$$p_{\theta}(\mathbf{x}_{t-1} \mid \mathbf{x}_t) = \mathcal{N}(\mathbf{x}_{t-1}; \boldsymbol{\mu}_{\theta}(\mathbf{x}_t, t), \sigma_t^2 \mathbf{I})$$

The mean $\boldsymbol{\mu}_{\theta}$ is parameterized by a neural network that takes the noisy input $\mathbf{x}_t$ and the timestep t , and predicts how to denoise it.

Why we need a neural network

While the forward process has a known, fixed form (just add Gaussian noise), the reverse process requires knowing $q(\mathbf{x}_{t-1} \mid \mathbf{x}_t)$ — which depends on the **entire data distribution**. Since we don't know this distribution analytically, we approximate it with a learned model p_{θ} .

The reverse process

U-Net: the noise prediction network

Architecture for predicting noise

The neural network $\epsilon_{\theta}(\mathbf{x}_t, t)$ that predicts noise uses a **U-Net** architecture (Ronneberger et al., 2015):

- **Encoder** (downsampling path): Progressively compresses the spatial dimensions while increasing channels
- **Bottleneck**: Processes the most compressed representation
- **Decoder** (upsampling path): Progressively restores spatial resolution
- **Skip connections**: Direct connections from encoder to decoder at each resolution level, preserving fine-grained details

Why U-Net?

The U-shape is ideal for denoising because it captures both **global structure** (via the bottleneck) and **local details** (via skip connections). The network needs both to effectively remove noise while preserving meaningful content.

U-Net architecture

Timestep conditioning

Telling the network what noise level to expect

The same U-Net must denoise at *every* timestep — from nearly clean ($t = 1$) to pure noise ($t = T$). It needs to know which timestep it's working at. The timestep t is converted to a vector using **sinusoidal embeddings** (the same idea as positional embeddings in transformers, Lecture 15):

$$\text{emb}(t) = [\sin(t/10000^{0/d}), \cos(t/10000^{0/d}), \sin(t/10000^{2/d}), \cos(t/10000^{2/d}), \dots]$$

This embedding is then added to or concatenated with the intermediate features inside the U-Net.

Timestep embedding

The training objective

Ho et al. (2020): predict the noise

Instead of predicting $\mathbf{x}_0$ or μ_θ directly, Ho et al. (2020) found it more effective to train the network to predict **the noise that was added**:

1. Sample a clean image $\mathbf{x}_0$ from the training data
2. Sample a random timestep $t \sim \text{Uniform}(1, T)$
3. Sample noise $\epsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$
4. Create the noisy version: $\mathbf{x}_t = \sqrt{\bar{a}_t} \mathbf{x}_0 + \sqrt{1 - \bar{a}_t} \epsilon$
5. Train the network to predict ϵ from $\mathbf{x}_t$ and t

The key insight

The network doesn't learn to generate images — it learns to **identify and remove noise**. Generation happens by applying this denoising repeatedly, starting from pure noise.

The training loop

The simplified loss

From ELBO to MSE

The full variational lower bound (ELBO) for diffusion models involves a sum of KL divergences across all timesteps. Ho et al. (2020) showed that a much simpler loss works just as well:

$$\mathcal{L}_{\text{simple}} = \mathbb{E}_{t, \mathbf{x}_0, \epsilon} \left[\|\epsilon - \epsilon_{\theta}(\mathbf{x}_t, t)\|^2 \right]$$

This is just **mean squared error** between the true noise ϵ and the predicted noise $\epsilon_{\theta}(\mathbf{x}_t, t)$.

Why simplicity wins

Dropping the weighting terms from the full ELBO loss actually *improves* sample quality. The uniform weighting across timesteps forces the network to denoise well at every noise level, rather than focusing on easy (low-noise) timesteps.

The simplified loss

Score matching perspective

Song & Ermon (2019)

An alternative view comes from **score matching** (Song & Ermon, 2019). The **score function** is the gradient of the log probability density:

$$\mathbf{s}(\mathbf{x}) = \nabla_{\mathbf{x}} \log p(\mathbf{x})$$

This vector field points "uphill" toward regions of high probability. If we know the score function, we can generate samples by starting from noise and following the gradient.

Connection to DDPM

It turns out that predicting noise ϵ is equivalent to estimating the score function: $\epsilon_{\theta}(\mathbf{x}_t, t) \approx -\sqrt{1 - \bar{\alpha}_t} \nabla_{\mathbf{x}_t} \log q(\mathbf{x}_t)$. The DDPM and score matching perspectives are mathematically unified by Song et al. (2021).

Score matching

DDPM sampling algorithm

Generating new samples

To generate a new sample, we run the learned reverse process:

1. Sample $\mathbf{x}_T \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$
2. For $t = T, T - 1, \dots, 1$:
 - Predict noise: $\hat{\epsilon} = \epsilon_{\theta}(\mathbf{x}_t, t)$
 - Compute: $\mathbf{x}_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left(\mathbf{x}_t - \frac{\beta_t}{\sqrt{1-\alpha_t}} \hat{\epsilon} \right) + \sigma_t \mathbf{z}$
 - where $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ for $t > 1$, else $\mathbf{z} = \mathbf{0}$
3. Return $\mathbf{x}_0$

The cost of quality

This requires **1000 forward passes** through the neural network (one per timestep). At ~100ms per pass on a GPU, that's ~100 seconds per image. This motivates faster sampling methods like DDIM.

The sampling process

DDIM: faster sampling

Song et al. (2021): Denoising Diffusion Implicit Models

DDIM makes sampling **deterministic** and allows **skipping steps**:

- Instead of 1000 steps, use a subsequence (e.g., 50 evenly spaced steps)
- The update rule becomes deterministic (no random noise $\mathbf{z}$)
- Same trained model — just a different sampling procedure

Comparison

Property	DDPM	DDIM
Steps needed	~1000	~20–50
Stochastic?	Yes (random noise added)	No (deterministic)
Sample quality	Excellent	Slightly lower
Speed	~100 seconds	~2–5 seconds
Same model?	Yes	Yes — no retraining needed

Diffusion vs autoregressive generation

Two paradigms for generation

	Autoregressive (Transformer)	Diffusion
Generation order	Left to right, one token at a time	All positions simultaneously, refining iteratively
Native domain	Discrete sequences (text)	Continuous signals (images, audio)
Steps to generate	N (sequence length)	T (denoising steps, typically 20–1000)
Key operation	Next-token prediction	Noise prediction
Training signal	Cross-entropy loss	MSE loss (noise prediction)

Not competitors — complements

Autoregressive models excel at **sequential, discrete** data (language). Diffusion models excel at **continuous, spatial** data (images, video, audio). As we'll see in Lecture 23, combining them yields the most powerful generative systems.

Diffusion vs autoregressive generation

Discussion

Questions to consider

1. **Iterative refinement:** Humans often create by revising drafts — a rough sketch becomes a polished drawing. Is diffusion's iterative denoising a better model of human creativity than autoregressive generation?
2. **The noise perspective:** Diffusion models learn by predicting noise. Transformers learn by predicting the next token. Are these fundamentally different, or two views of the same underlying process?
3. **Information destruction:** The forward process deliberately destroys all information. Why does this make the reverse process *easier* to learn, rather than harder?
4. **Computational cost:** DDPM requires ~1000 neural network

Further reading

Further reading

Ho, Jain & Abbeel (2020, *NeurIPS*) "Denoising Diffusion Probabilistic Models" — The paper that made diffusion models practical. Clean formulation, excellent results.

Sohl-Dickstein et al. (2015, *ICML*) "Deep Unsupervised Learning using Nonequilibrium Thermodynamics" — The original diffusion model paper, grounded in statistical physics.

Song & Ermon (2019, *NeurIPS*) "Generative Modeling by Estimating Gradients of the Data Distribution" — Score matching perspective on diffusion.

Nichol & Dhariwal (2021, *ICML*) "Improved Denoising Diffusion Probabilistic Models" — Cosine schedule, learned variance, improved sampling.

Song et al. (2021, *ICLR*) "Denoising Diffusion Implicit Models" — DDIM: deterministic, fast sampling from the same trained model.

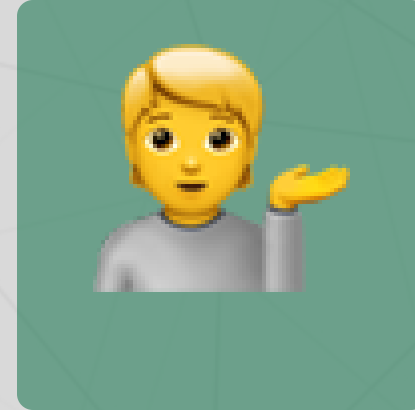
Questions?



Email



Discord



Office hours

Up next...

Diffusion model extensions: latent diffusion, classifier-free guidance, and the Diffusion Transformer